Investigación: detección del personal con Jetson TX2 + cámara IP

1. Confirmación de la arquitectura

Tu esquema es correcto, con una aclaración: **son dos formas de usar el mismo sistema, y la laptop cumple dos roles distintos.**

Cámara IP xView ──(red / RTSP)──► Orbitty + Jetson TX2 ──► resultado (detecciones, alertas, registros)

▲
│ SSH / monitor / flasheo
Laptop con Ubuntu

Modo de operación normal (producción):
Cámara IP → Orbitty con los programas instalados. La Orbitty es la que **procesa todo**: recibe el video, corre YOLOv5 en su GPU y aplica tu lógica. La laptop **no hace falta** una vez que todo está instalado.

Rol de la laptop con Ubuntu (desarrollo):

1. **Flashear** el Jetson (instalar el sistema operativo y CUDA desde la laptop).
2. **Controlarlo y programarlo** a distancia por SSH.
3. **Ver resultados** o probar código antes de pasarlo a la Orbitty.

Entonces tu idea de "instalar Ubuntu como ISO en la Orbitty" se corrige así: **el Jetson no se instala con una ISO**; se flashea desde la laptop con NVIDIA SDK Manager, que le pone un Ubuntu especial (L4T) con CUDA incluido. Después de eso el Jetson funciona solo.

2. Qué es PyTorch y qué es CUDA

	Qué es	Para qué sirve	¿Obligatorio?
PyTorch	Framework de inteligencia artificial en Python	YOLOv5 está escrito con él; carga el modelo y hace las predicciones	**Sí.** Sin PyTorch, YOLOv5 no corre
CUDA	Plataforma de NVIDIA para calcular en la GPU	Permite que PyTorch use la GPU del Jetson en vez de la CPU	**Técnicamente no, en la práctica sí**

Sin CUDA, YOLOv5 funciona solo en CPU, pero en el TX2 daría aproximadamente 1 FPS o menos, algo inviable para video en tiempo real. Con CUDA, el modelo chico (yolov5s) puede dar un video fluido.

3. Qué tenés hoy y qué falta

Con solo el hardware y Python 3.6.8, lo más probable es que **falte** casi todo:

Componente	Estado probable
JetPack (Ubuntu L4T + CUDA + cuDNN + TensorRT + OpenCV)	Hay que verificar o flashear
PyTorch para Jetson	Falta
torchvision	Falta
YOLOv5	Falta
OpenCV con GStreamer (para leer la cámara)	Viene con JetPack

Para verificar qué tenés en la Orbitty:

```
cat /etc/nv_tegra_release      # versión de L4T/JetPack
nvcc --version                 # CUDA instalado
python3 --version
python3 -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

4. Plan de instalación paso a paso

1. **Flashear** desde la laptop (Ubuntu 18.04 o 20.04) con **JetPack 4.6** y el **BSP de Connect Tech** correspondiente a tu Orbitty. Revisá en la web de Connect Tech cuál es el paquete para tu modelo, porque la placa portadora necesita su propio soporte.
2. **Instalar PyTorch** usando el wheel (.whl) que NVIDIA publica para Jetson. **No uses pip install torch**, porque en ARM instala una versión sin GPU. Para Python 3.6, el máximo es **PyTorch 1.10**.
3. **Compilar torchvision** desde el código fuente, en la versión compatible con PyTorch 1.10.
4. **Instalar YOLOv5 v6.0**: según recuerdo, es la última versión que acepta Python 3.6, pero conviene confirmarlo en el repositorio de Ultralytics antes de instalar. Las versiones más nuevas piden Python 3.7 o superior.
5. **Probar con un video** y recién después con la cámara.
6. **Optimizar con TensorRT** (exportar el modelo), lo que suele aumentar mucho los FPS en el TX2.

5. Cámara IP xView

- Al ser **antigua**, lo primero es comprobar que soporte **RTSP**, que es lo que se usa para sacar el video por red. Revisá el manual o su panel web. La URL suele tener esta forma (varía por modelo):
  rtsp://usuario:clave@IP_CAMARA:554/ruta

- Si solo ofrece **MJPEG** por HTTP, también sirve, pero con más consumo de red.
- La cámara y la Orbitty deben estar en la **misma red** (cable o switch), con IP fija o reservada.
- Prueba rápida antes de programar: abrir la URL con VLC desde la laptop.
- Lectura desde Python con OpenCV:

```
import cv2
cap = cv2.VideoCapture("rtsp://usuario:clave@192.168.1.50:554/stream", cv2.CAP_FFMPEG)
ok, frame = cap.read()
```

- Una cámara antigua puede tener baja resolución o mala imagen con poca luz, lo que afecta la precisión de la detección.

6. La lógica de "personal de la tienda"

Esto es lo más importante a definir, porque **YOLOv5 preentrenado solo detecta "persona"**, no distingue empleado de cliente. Opciones, de más simple a más compleja:

1. **Zonas de la imagen:** definir áreas (detrás del mostrador, caja) y considerar personal a quien esté allí. Simple pero poco confiable.
2. **Color del uniforme:** detectar la persona con YOLOv5 y analizar el color predominante del torso con OpenCV. Funciona bien si el uniforme es distintivo.
3. **Modelo propio:** entrenar YOLOv5 con una clase "empleado" y otra "cliente", usando imágenes de tu propia tienda. Es la más precisa, pero requiere recolectar y etiquetar imágenes, y entrenar en una PC con GPU (no en el TX2).
4. **Reconocimiento facial:** posible, pero con implicancias de privacidad y mayor costo de procesamiento. En Argentina conviene revisar la Ley de Protección de Datos Personales antes de usarlo.

7. Limitaciones a tener presentes

- El TX2 queda limitado a **JetPack 4.x, Ubuntu 18.04 y Python 3.6**. Si más adelante necesitan software más nuevo, habría que migrar a un Jetson más moderno (Xavier u Orin).
- Esperá entre unos pocos y unas decenas de FPS según el modelo y la optimización; para contar o identificar personal en una tienda suele ser suficiente.
- Usá yolov5s o yolov5n, no los modelos grandes.

8. Resumen

Pregunta	Respuesta
¿Orbitty → cámara IP funciona sola?	Sí, una vez instalado todo, sin laptop
¿Para qué sirve la laptop?	Flashear, programar por SSH, entrenar o probar

¿Ubuntu por ISO?	No, se flashea con JetPack (SDK Manager)
¿PyTorch necesario?	Sí
¿CUDA necesario?	Para rendimiento real, sí
¿Versión de YOLOv5?	v6.0 (a verificar) con PyTorch 1.10